

Deployment Checklist — Nucleus AI

Pre-Deployment Checklist

Environment

- [] `.env.production` created with all required variables
- [] All `CHANGE_ME` values replaced with real values
- [] `POSTGRES_PASSWORD` is strong (32+ chars, random)
- [] `SECRET_KEY` generated (`openssl rand -hex 64`)
- [] `NEXTAUTH_SECRET` generated (`openssl rand -base64 32`)
- [] `OPENAI_API_KEY` is valid and has sufficient quota
- [] `DOMAIN` set to actual domain name
- [] `CORS_ORIGINS` matches production domain
- [] `DEBUG` is set to `false`

Infrastructure

- [] Server/cloud resources provisioned
- [] DNS records pointing to server IP
- [] SSL certificate obtained (Let's Encrypt or managed)
- [] Firewall configured (ports 80, 443, 22 only)
- [] SSH key-based authentication enabled
- [] Docker and Docker Compose installed

Database

- [] PostgreSQL accessible from backend
- [] Database and user created
- [] Migrations applied (`alembic upgrade head`)
- [] Prisma migrations applied (`npx prisma migrate deploy`)
- [] Backup schedule configured

Vector Database

- [] Qdrant accessible from backend
- [] API key configured (if using Qdrant Cloud)
- [] Collection will be auto-created on first use

Nginx

- [] `server_name` updated in nginx config
 - [] SSL certificates placed in `nginx/ssl/`
 - [] Certificate auto-renewal configured
-

Deployment Steps

```
# 1. Clone/update code
git clone https://github.com/your-org/nucleus-ai.git /opt/nucleus-ai
cd /opt/nucleus-ai

# 2. Configure environment
cp .env.production.example .env.production
nano .env.production

# 3. Update nginx domain
sed -i 's/yourdomain.com/ACTUAL_DOMAIN/g' nginx/conf.d/default.conf

# 4. Build and start
docker compose -f docker-compose.prod.yml --env-file .env.production up -d --build

# 5. Run migrations
docker compose -f docker-compose.prod.yml exec backend alembic upgrade head
docker compose -f docker-compose.prod.yml exec frontend npx prisma migrate deploy

# 6. Verify
curl -s https://yourdomain.com/health
```

Post-Deployment Verification

- [] `https://yourdomain.com` loads the frontend
- [] `https://yourdomain.com/health` returns `{"status": "healthy"}`
- [] `https://yourdomain.com/api/v1/health` returns healthy
- [] User registration/login works
- [] Brand profile creation works
- [] Context ingestion works (file upload)
- [] AI workflow execution returns results
- [] Attribution dashboard loads data
- [] SSL certificate valid (check with `curl -vI`)
- [] All containers healthy (`docker compose ps`)
- [] Logs show no errors (`docker compose logs --tail=50`)

Rollback Procedure

If deployment fails:

```
# 1. Quick rollback to previous image
docker compose -f docker-compose.prod.yml --env-file .env.production down
git checkout HEAD~1
docker compose -f docker-compose.prod.yml --env-file .env.production up -d --build

# 2. If database migration caused issues
docker compose -f docker-compose.prod.yml exec backend alembic downgrade -1

# 3. Full rollback from backup
./scripts/deployment/restore.sh /opt/backups/nucleus-ai/db_LATEST.sql.gz
```

Troubleshooting Guide

Container Issues

Symptom	Check	Fix
Container keeps restarting	<code>docker compose logs <service></code>	Fix config errors
Health check failing	<code>curl localhost:8000/health</code>	Check application logs
Out of memory	<code>docker stats</code>	Increase memory limits
Image build fails	Check Dockerfile syntax	Ensure all COPY paths correct

Network Issues

Symptom	Check	Fix
502 Bad Gateway	Backend running?	<code>docker compose ps</code>
Connection refused	Firewall rules	<code>ufw status</code>
SSL errors	Certificate files exist?	Check <code>nginx/ssl/</code>
CORS errors	Check <code>CORS_ORIGINS</code> env	Must match exact domain

Database Issues

Symptom	Check	Fix
Connection refused	PostgreSQL running?	<code>docker compose exec postgres pg_isready</code>
Authentication failed	Check <code>DATABASE_URL</code>	Verify user/password
Migration errors	<code>alembic current</code>	<code>alembic upgrade head</code>
Slow queries	<code>pg_stat_activity</code>	Add indexes, check pool

Common Fixes

```
# Restart all services
docker compose -f docker-compose.prod.yml restart

# Rebuild from scratch
docker compose -f docker-compose.prod.yml down
docker compose -f docker-compose.prod.yml up -d --build --force-recreate

# Clean up Docker resources
docker system prune -a --volumes # WARNING: removes all unused data

# Check resource usage
docker stats --no-stream
df -h # Disk space
free -h # Memory
```